



CHUYÊN ĐỀ GAME ONLINE

BÀI 3: ĐỒNG BỘ HÓA CHUYỂN ĐỘNG VÀ CAMERA TRONG SHARED MODE

- ☐ Đồng bộ hóa chuyển động người chơi giữa các client.
- ☐ Điều khiển di chuyển nhân vật trong mạng
- ☐ Thiết lập camera theo dõi người chơi trong môi trường mạng.
- ☐ Sử dụng Cinemachine trong mạng

Trong Fusion, code cập nhật trên mỗi tick, ví dụ như xử lý chuyển động, chúng ta không nên viết trong Update/FixedUpdate như thông thường.

Thay vào đó, chúng ta sẽ viết trong FixedUpdateNetwork, điều này đảm bảo chuyển động được mượt mà trên tất cả client

FixedUpdateNetwork được gọi theo mỗi tick mạng, đảm bảo rằng:

- ❑ Logic chuyển động được thực hiện đồng bộ trên tất cả các client và server.
- ❑ Chuyển động được tính toán tại các thời điểm chính xác trong mô hình tick-based của Fusion.

Giảm lỗi đồng bộ thời gian

- ❑ Khi sử dụng Update(), tốc độ khung hình khác nhau giữa các client có thể làm cho chuyển động không nhất quán.
- ❑ FixedUpdateNetwork tách biệt logic chuyển động khỏi tốc độ khung hình, mọi hành động di chuyển được xử lý dựa trên thời gian tick của mạng thay vì thời gian khung hình.

Trước tiên, chúng ta thêm các component cần thiết vào trong Player prefab đã có ở bài trước

- ☐ CharacterController.
- ☐ NetworkObject.
- ☐ NetworkTransform.

NetworkObject là thành phần bắt buộc để một đối tượng tồn tại và tương tác trong môi trường mạng.

- ❑ Cấp danh tính duy nhất cho mỗi đối tượng trên mạng (Network ID).
- ❑ Theo dõi quyền kiểm soát (Ownership) của từng đối tượng.
- ❑ Cho phép các hành động mạng như spawn, destroy, và chuyển quyền kiểm soát.

Quyền kiểm soát trong NetworkObject

- ☐ Object.HasStateAuthority: Kiểm tra quyền kiểm soát trạng thái.
- ☐ Object.HasInputAuthority: Kiểm tra quyền điều khiển đầu vào.

NetworkTransform là thành phần dùng để đồng bộ hóa vị trí, hướng, và tỉ lệ của đối tượng trên các client.

- ❑ Đảm bảo đối tượng xuất hiện nhất quán trên tất cả các client.
- ❑ Hỗ trợ tùy chỉnh tốc độ và mức độ chính xác của đồng bộ hóa.

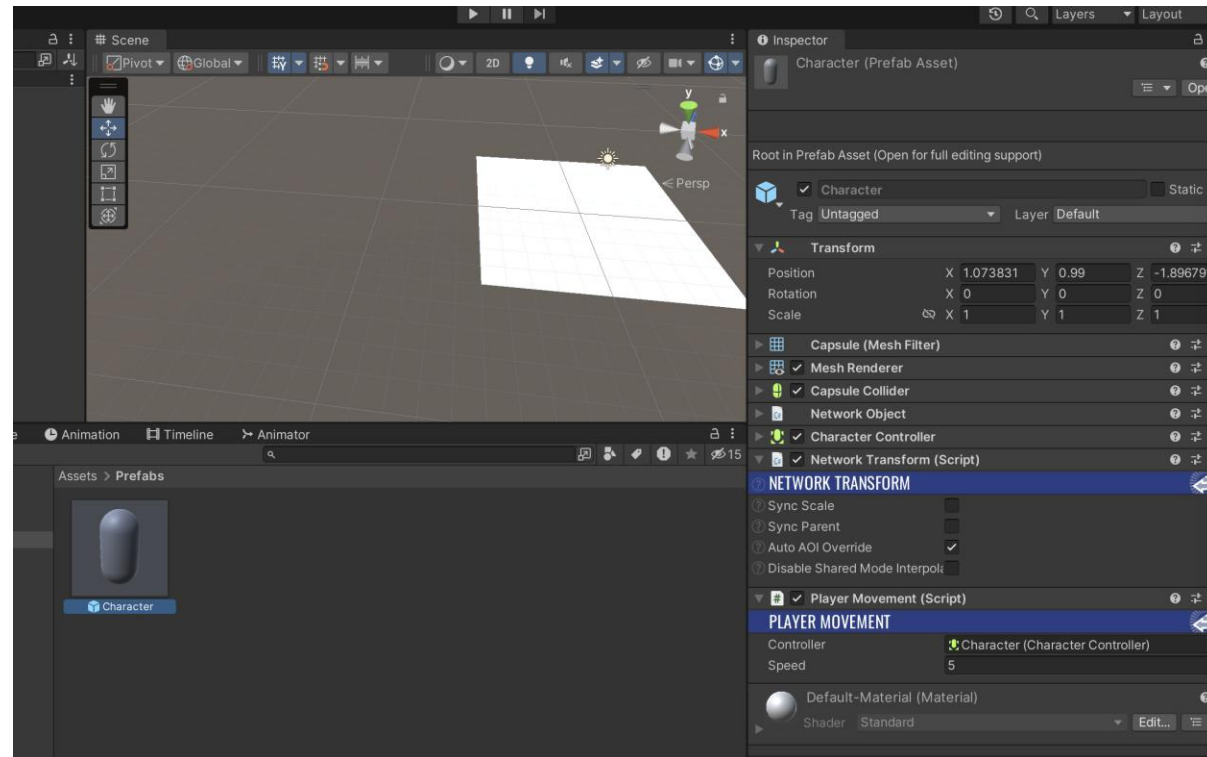
Tiếp theo, chúng ta viết script PlayerMovement, chi tiết như sau

```
using UnityEngine;
using Fusion;

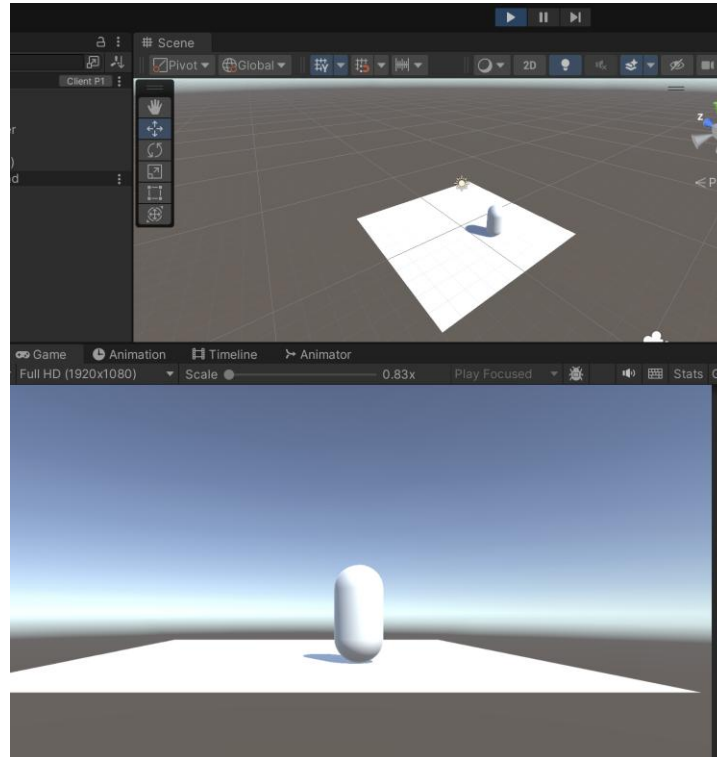
public class PlayerMovement : NetworkBehaviour
{
    public CharacterController controller;
    public float speed = 5f;

    public override void FixedUpdateNetwork()
    {
        var horizontal = Input.GetAxis("Horizontal");
        var vertical = Input.GetAxis("Vertical");
        Vector3 move = new Vector3(horizontal, 0, vertical);
        controller.Move(move * speed * Runner.deltaTime);
    }
}
```

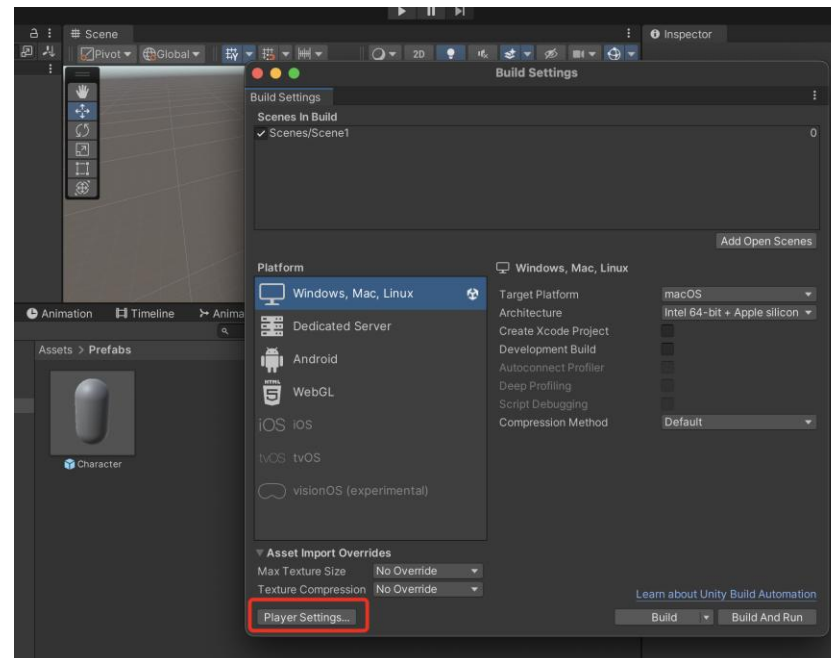
Thêm script vừa viết vào Player prefab như ảnh bên dưới



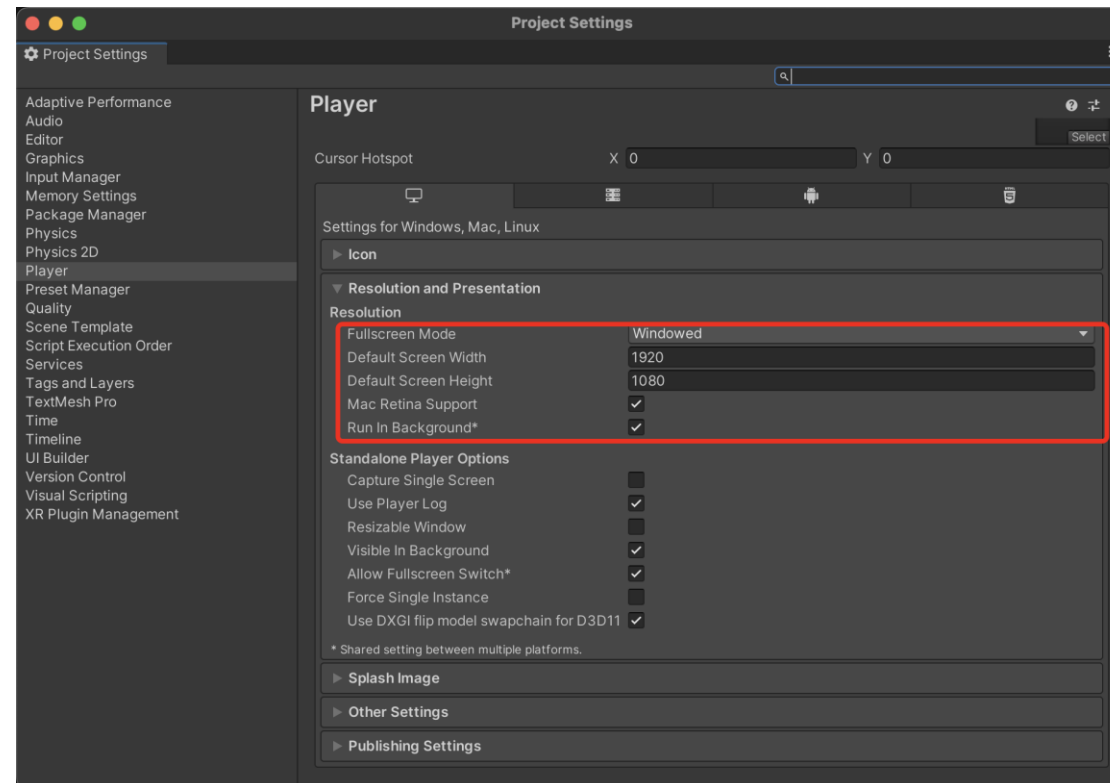
Tiến hành chạy game và điều khiển chuyển động của nhân vật trong mạng



Để có thể chạy được hai đối tượng chơi trong mạng, chúng ta vào File, chọn Build Settings, thêm scene hiện tại vào danh sách. Sau đó chọn Player Settings ở góc trái bên dưới



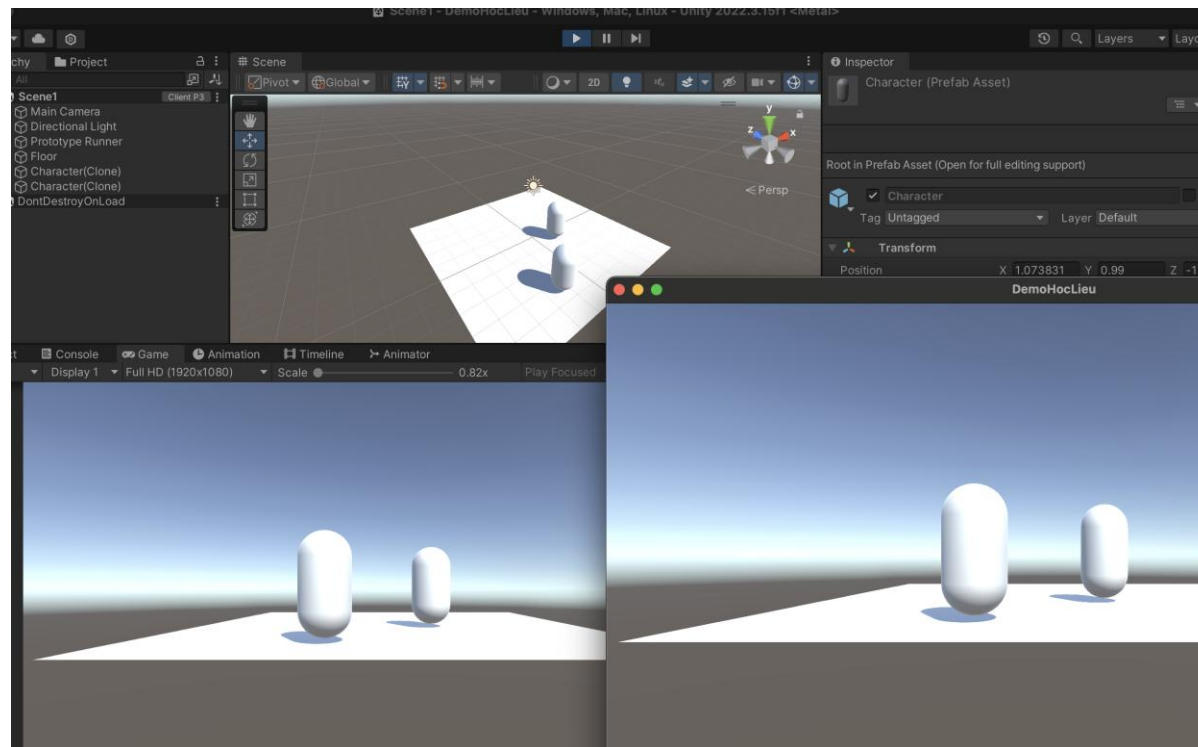
Trong cửa sổ Player Settings, ở mục Resolution and Presentation, tiến hành cài đặt như ảnh



Sau đó, chúng ta chọn File, Build And Run, Unity sẽ tiến hành build và chạy game ở chế độ cửa sổ.

Tiếp theo, chúng ta chạy game trong cửa sổ của Unity. Như vậy, đã có hai cửa sổ game cùng chạy và tạo đối tượng vào mạng.

Bên dưới là hình ảnh của hai nhân vật được tạo ra trong mạng và điều khiển di chuyển độc lập



Khi điều khiển chuyển động, chúng ta cần chú ý
Quyền kiểm soát đầu vào cho từng Player.

Chỉ xử lý đầu vào trên client có quyền điều khiển.

```
public override void FixedUpdateNetwork()  
{  
    if (!Object.HasInputAuthority) return;  
    var horizontal = Input.GetAxis("Horizontal");  
    var vertical = Input.GetAxis("Vertical");  
}
```

Cinemachine là công cụ mạnh mẽ trong Unity để thiết lập và quản lý camera. Khi kết hợp với Photon Fusion 2, bạn cần đảm bảo camera theo dõi đúng người chơi mà client điều khiển, đồng thời tránh xung đột camera giữa các client.

Các bước khi áp dụng Cinemachine

- ☐ Thêm Cinemachine vào dự án
- ☐ Thiết lập Cinemachine Virtual Camera
- ☐ Tự động gán camera khi Player spawn
- ☐ Đảm bảo camera chỉ hoạt động trên Player của client

Thêm Cinemachine Virtual Camera vào trong scene như, chúng ta cài đặt các thuộc tính sau

- ☐ Follow: Gán Transform của Player.
- ☐ LookAt: Gán Transform của Player (nếu cần).
- ☐ Body: Set thành Framing Transposer để camera theo dõi vị trí.
- ☐ Aim: Set thành Hard Lock to Target hoặc Composer.

Thêm script CameraFollow và gắn vào Virtual Camera vừa tạo

```
public class CameraFollow : MonoBehaviour
{
    public CinemachineVirtualCamera virtualCamera;

    public void AssignCamera(Transform playerTransform)
    {
        virtualCamera.Follow = playerTransform;
        virtualCamera.LookAt = playerTransform;
    }
}
```

Đảm bảo Player Prefab có NetworkObject và Transform để gán vào Cinemachine. Chúng ta tạo class PlayerSetup

```
public class PlayerSetup : NetworkBehaviour
{
    public void SetupCamera()
    {
        if (Object.HasInputAuthority)
        {
            CameraFollow cameraFollow = FindObjectOfType<CameraFollow>();
            if (cameraFollow != null)
            {
                cameraFollow.AssignCamera(transform);
            }
        }
    }
}
```

Khi một Player được spawn, camera phải tự động cập nhật Follow và LookAt

```
using Fusion;
public class PlayerSpawner : SimulationBehaviour, IPlayerJoined
{
    public GameObject PlayerPrefab;

    public void PlayerJoined(PlayerRef player)
    {
        if (player == Runner.LocalPlayer)
        {
            Runner.Spawn(PlayerPrefab, new Vector3(0,0,0), Quaternion.identity,
                Runner.LocalPlayer, (runner, obj) =>
            {
                var _player = obj.GetComponent<PlayerSetup>();
                _player.SetupCamera();
            });
        }
    }
}
```

- Điều chỉnh Cinemachine Virtual Camera
- Framing Transposer (Body):
 - ✓ Điều chỉnh khoảng cách camera với Player.
 - ✓ Dead Zone Width/Height: Tránh camera chuyển động liên tục khi Player di chuyển nhỏ.
- Composer (Aim):
 - ✓ Đặt vị trí camera phù hợp với góc nhìn mong muốn.

Ví dụ: Game Đua Xe

- ☐ Follow: Camera theo sau xe của người chơi.
- ☐ LookAt: Hướng về phía trước xe.
- ☐ Cấu hình: Sử dụng Framing Transposer với góc nhìn nghiêng để dễ quan sát đường đua.

Ví dụ: Game Hành Động (RPG)

- ☐ Follow: Camera di chuyển theo nhân vật chính.
- ☐ LookAt: Hướng vào nhân vật.
- ☐ Cấu hình: Composer để điều chỉnh góc camera.

- ❑ Dead Zone: Thiết lập vùng mà camera không di chuyển khi Player di chuyển nhẹ. Giảm rung lắc camera khi Player đứng yên.
- ❑ Soft Zone: Tạo vùng mượt khi camera chuyển động ra khỏi trung tâm.
- ❑ Damping: Làm mượt các thay đổi vị trí hoặc hướng của camera.

➤ Điều chỉnh khoảng cách camera động

Cho phép người chơi thay đổi khoảng cách camera bằng Input, ví dụ như scroll chuột

```
public class CameraFollow : MonoBehaviour

{
    private CinemachineFramingTransposer framingTransposer;

    private void Start()
    {
        framingTransposer = virtualCamera.GetCinemachineComponent<CinemachineFramingTransposer>();
    }

    private void Update()
    {
        float scroll = Input.GetAxis("Mouse ScrollWheel");
        framingTransposer.m_CameraDistance -= scroll;
        framingTransposer.m_CameraDistance = Mathf.Clamp(framingTransposer.m_CameraDistance, 2f, 10f);
    }
}
```

- ☐ Thực hiện việc đồng bộ hóa chuyển động người chơi giữa các client.
- ☐ Cách thức để điều khiển di chuyển nhân vật trong mạng
- ☐ Sử dụng Cinemachine trong mạng



Kết thúc